

THUNDER — MASTER PLANNING PROMPT

You are the principal product engineer and AI systems architect for a solo hackathon project called **Thunder**.

This is a time-constrained implementation task. I am one developer with approximately five remaining development hours. Your responsibility is to design the smallest polished, credible, working vertical slice that can compete for a top-two position.

CRITICAL FIRST INSTRUCTION

Do not write, create, edit, or delete any application code yet.

First:

1. Inspect the entire repository.
2. Determine its current state.
3. Research current official documentation only where necessary.
4. Produce a final implementation plan for my approval.
5. Identify every proposed dependency, API call, file, route, schema, graph node, and user-visible screen.
6. Explicitly identify what will be excluded.
7. Wait for my approval before implementation.

Do not ask broad product questions. Make sensible decisions from this specification. Ask only if a genuinely blocking technical fact cannot be inferred.

1. PRODUCT DEFINITION

Thunder is:

A pre-publication audience intelligence and scenario-testing environment for creators.

Primary tagline:

Test your post before your audience does.

Thunder is not:

- A new social network
- An Instagram clone
- A generic caption generator
- A scheduler

- A follower-management application
- A direct publishing tool
- A fake exact view predictor

The main user is a creator, founder, educator, marketer, or technical communicator who has historical audience comments but does not know how a new draft will land with different parts of that audience.

2. CORE PRODUCT WORKFLOW

The complete vertical slice must be:

```
Historical audience comments
↓
Evidence-backed audience model
↓
Three distinct audience segments
↓
Creator's draft post
↓
Multi-agent scenario testing
↓
Segment-specific reactions and disagreement
↓
Transparent deterministic diagnostics
↓
Safety and credibility guardrails
↓
Improved five-slide carousel
↓
Before-versus-after comparison
↓
Evidence explaining every major recommendation
```

The workflow must feel like a coherent product, not a collection of unrelated AI cards.

3. CORE DEMO STORY

The default demo should work immediately using high-quality seeded example data.

Suggested scenario:

A technical creator wants to publish a carousel explaining AI agents or MCP.

Their historical comments contain patterns such as:

- Beginners asking for simple explanations
- Developers asking for code and implementation detail
- Experienced practitioners challenging oversimplified claims
- People asking where to begin
- People worrying about hype, privacy, or unsupported promises

The creator submits a draft that is too broad and slightly exaggerated.

Thunder constructs an audience model, runs grounded scenario testing, exposes disagreement, identifies weaknesses, and produces a materially better five-slide carousel.

The user must also be able to replace the seeded comments and draft with their own text.

4. REAL AGENTIC SCENARIO TESTING

Thunder must contain a genuine multi-step agentic workflow.

Do not implement “agents” as merely three static persona cards or three unrelated prompts.

Use **LangGraph.js** or an equally defensible TypeScript-native graph orchestration approach.

The agents should share typed state, perform distinct responsibilities, and produce auditable outputs.

Use a small, controlled graph rather than uncontrolled autonomous agents.

Proposed graph responsibilities:

A. Audience Research Agent

Responsibilities:

- Analyse historical comments
- Discover recurring needs, expertise levels, objections, motivations, and preferred content patterns
- Produce exactly three differentiated audience segments
- Attach valid historical comment IDs as evidence
- Avoid fabricating evidence
- Express uncertainty where support is weak

B. Scenario Simulation Agent

Responsibilities:

- Take the audience model and creator draft
- Simulate how each segment interprets the draft
- Produce different, sometimes conflicting reactions
- Identify what each segment understood
- Identify what each segment valued
- Identify what each segment disliked or distrusted
- Identify missing information
- Estimate likely action using a constrained enum
- Produce qualitative factor ratings for later deterministic scoring

C. Adversarial Critic / Guardrail Agent

Responsibilities:

- Challenge the draft and simulated reactions
- Detect exaggerated claims
- Detect missing context
- Detect unsupported conclusions
- Detect manipulative wording
- Detect privacy or safety concerns
- Detect likely misinterpretations
- Flag weak or fabricated evidence
- Never pretend to establish objective truth

D. Content Strategy Agent

Responsibilities:

- Resolve trade-offs between conflicting audience needs
- Preserve the creator's central message
- Produce a stronger hook
- Produce exactly five carousel slides
- Produce a caption
- Produce a CTA
- Explain significant changes
- Reference evidence and diagnostic findings
- Avoid optimizing by making unsupported claims

E. Verification Node

Prefer deterministic TypeScript validation rather than another expensive LLM call.

Responsibilities:

- Confirm all evidence IDs exist in the imported source comments
- Confirm exactly three audience segments exist
- Confirm exactly five slides exist
- Confirm scores are within allowed bounds
- Confirm required fields are present
- Confirm unsupported IDs are rejected or repaired
- Confirm the final result conforms to the Zod schemas

The final plan may combine some LLM responsibilities into fewer model calls to reduce latency and implementation time, but the product must preserve the clearly separated agent responsibilities and visible scenario-testing flow.

Explain exactly which graph nodes use LLM calls and which use deterministic code.

5. CREDIBILITY REQUIREMENTS

This is the most important product requirement.

Thunder must not look like “ChatGPT with pretty cards.”

Evidence grounding

Every audience segment must show the historical comments supporting it.

Each imported comment should receive a stable ID such as:

```
C01
C02
C03
```

Any evidence ID produced by the model must be validated against the actual imported IDs.

If a model returns a nonexistent evidence ID:

- Reject it
- Repair it
- Retry the relevant structured call once
- Or remove the unsupported claim

Never silently display fabricated evidence.

Honest product claims

Use wording such as:

“Thunder runs a grounded scenario simulation based on patterns in the creator’s supplied audience data.”

Do not claim:

- It predicts real humans perfectly
- It knows exactly how many views a post will receive
- It guarantees virality
- Simulated people are actual users
- Generated scores are objective scientific measurements

Confidence

Display confidence as:

- Low
- Medium
- High

Confidence should be based on evidence quantity, consistency, and segment support—not invented directly by the model without explanation.

6. DIAGNOSTICS AND DETERMINISTIC SCORING

The LLM may assess tightly defined qualitative factors, but it must not invent the final overall scores.

The TypeScript application must compute final scores using visible formulas and weights.

Required diagnostics:

- Audience fit
- Clarity
- Save potential
- Discussion potential
- Misinterpretation risk

Possible factor inputs:

- Hook strength
- Readability
- Specificity

- Practical usefulness
- Segment relevance
- Evidence support
- Novelty
- Question potential
- Controversy risk
- Ambiguity
- Exaggeration
- Missing context

All factor values must be constrained, for example integers from 0 to 10.

The plan must define understandable scoring formulas.

Example structure:

```
clarity =
  0.30 × readability
+ 0.25 × specificity
+ 0.25 × structure
+ 0.20 × context completeness
```

Round and clamp all final scores to 0–100.

The UI should provide a small “How this was calculated” disclosure or tooltip.

Do not implement a fake exact views number.

Optional descriptive output:

- Relative performance: Low / Moderate / Strong
- Primary likely strength
- Primary likely weakness
- Confidence level

7. REQUIRED USER EXPERIENCE

The application must be visually polished enough for a live hackathon demonstration.

Use a modern, premium creator-tool aesthetic—not a generic admin dashboard and not an Instagram clone.

Prioritize:

- Strong typography
- Spacious layout
- Clear hierarchy
- Responsive desktop presentation
- Tasteful motion
- Skeleton/loading states
- Useful empty states
- Clear error handling
- A cohesive visual system
- No broken controls
- No placeholder lorem ipsum
- No generic AI-generated gradient overload

Recommended application structure

A single-page guided workspace or a small number of focused views is preferable.

Potential stages:

Stage 1 — Audience Data

- Seed demo button
- Historical comment input
- Creator context input
- Draft post input
- Clear example content
- “Run Audience Test” primary CTA

Stage 2 — Audience Twin

- Exactly three segment cards
- Segment name
- Description
- Needs
- Frustrations
- Expertise level
- Evidence count
- Confidence
- Expandable evidence comments

Stage 3 — Reaction Lab / Audience Jury

- Three segment-specific reactions
- Clearly visible disagreement
- What each understood
- What each liked

- What each challenged
- Missing information
- Likely action
- Optional “Poke this segment” only if core work is complete

Stage 4 — Diagnostics and Guardrails

- Five transparent scores
- Key strengths
- Key weaknesses
- Guardrail findings
- Visible scoring explanation

Stage 5 — Optimized Carousel

- Better hook
- Exactly five slide previews
- Caption
- CTA
- Why it changed
- Evidence references

Stage 6 — Before and After

This must be a memorable demo moment.

Show original versus optimized diagnostics:

	Original	Optimized
Audience fit	62	84
Clarity	48	81
Save potential	41	76
Misinterpretation risk	67	29

Actual scores must come from the implemented deterministic formulas.

Do not hard-code fake score improvements in the live workflow.

8. TECHNICAL STACK

Use a single full-stack TypeScript repository.

Preferred baseline:

- Current stable Next.js with App Router
- TypeScript with strict mode
- React
- Tailwind CSS
- shadcn/ui or carefully selected lightweight components
- Zod for every AI and API boundary
- LangGraph.js for orchestration
- OpenAI API with structured outputs
- Next.js Route Handlers as backend
- Deterministic scoring utilities
- Local seeded demo data
- Mock fallback mode
- Deployable on Vercel or Render

Do not add a separate FastAPI or Python service.

Do not add:

- Authentication
- Database
- User accounts
- Followers
- Friends
- Social graph
- Direct posting
- WebSockets
- Microservices
- Separate backend repository
- Unnecessary state-management libraries
- Premature abstractions

Use environment variables for secrets.

Create an `.env.example`.

Never expose an API key to the browser.

9. MODEL AND API STRATEGY

Use the strongest available OpenAI reasoning/coding-capable model suitable for the task, but optimise architecture for:

- Reliability

- Structured output compliance
- Latency
- Cost
- Hackathon stability

The implementation should ideally use approximately two core model calls:

Core model call 1

May produce:

- Audience segments
- Evidence references
- Segment scenario reactions
- Constrained diagnostic factor ratings
- Confidence inputs
- Guardrail findings

Deterministic processing

Then calculate:

- Final diagnostic scores
- Confidence
- Evidence validity
- Aggregations
- Risk levels

Core model call 2

May produce:

- Improved hook
- Five-slide carousel
- Caption
- CTA
- Change explanations
- Evidence references

The final planning document must decide whether this two-call design is feasible with LangGraph while maintaining distinct agent responsibilities.

Use structured outputs or strict JSON schemas—not free-form parsing.

Every AI response must have:

- A Zod schema

- Graceful parse failure handling
 - One bounded retry or repair strategy
 - A mock fallback
-

10. MOCK FALLBACK

The entire application must remain demonstrable if:

- OpenAI fails
- Internet connectivity is unstable
- Rate limits occur
- Environment variables are missing
- An output fails validation

Implement a clearly controlled demo/mock fallback.

Requirements:

- Seeded default input
- Seeded complete output matching the real schemas
- Visible but unobtrusive “Demo data” or “Fallback mode” indicator
- No crash
- No blank screen
- Same UI path as the live workflow

The core UI should be buildable and testable using the mock result before live AI wiring.

11. OPTIONAL SPONSOR INTEGRATION

Only after the complete core workflow works.

Preferred optional integration:

fal.ai — Generate Carousel Cover

- Separate button
- Uses the optimized hook and creator context
- Generates one carousel cover
- Non-blocking
- Failure does not damage the workflow
- Existing designed fallback cover always remains available

Alternative only if n8n access becomes available easily:

n8n — Approved Creator Workflow Export

- User explicitly approves optimized content
- Export the approved content to an n8n webhook
- Do not directly publish to a real social platform
- Keep it isolated from core analysis

Do not make n8n a dependency.

Skip Firecrawl and ElevenLabs unless all required features, deployment, testing, and pitch materials are already complete.

12. DEVELOPMENT ORDER

The final implementation plan should optimise for a solo developer with limited time.

Use approximately this sequence unless repository inspection provides a strong reason to adjust it:

Phase 1 — Foundation

- Initialise or repair Next.js project
- Install minimal dependencies
- Establish design tokens and page shell
- Define types and Zod schemas
- Add seeded comments, creator context, draft, and complete mock response

Phase 2 — Complete Mock Vertical Slice

- Build the entire end-to-end user experience against mock data
- Ensure every required stage renders
- Add loading, errors, empty states, and before/after view
- Verify responsive layout

Phase 3 — Deterministic Engine

- Comment ID normalization
- Evidence validator
- Diagnostic formulas
- Confidence calculation
- Output verification
- Unit tests for pure utilities

Phase 4 — Live Agentic Workflow

- Add OpenAI client
- Build LangGraph state and nodes
- Add structured output calls
- Add one bounded retry
- Add fallback behavior
- Connect `/api/analyze`

Phase 5 — Polish and Reliability

- Browser-test the complete demo
- Fix console errors
- Run lint
- Run typecheck
- Run tests
- Run production build
- Improve visual clarity and interaction
- Ensure demo can be reset instantly

Phase 6 — Optional Sponsor Feature

- fal.ai cover generation if time remains
- Otherwise n8n export if access is available and integration is trivial

Phase 7 — Submission Readiness

- README
- Architecture diagram in Mermaid
- Setup instructions
- `.env.example`
- Product explanation
- Honest limitations
- Demo script
- Deployment configuration

The repository must remain runnable after every phase.

13. ENGINEERING QUALITY

Use:

- Explicit TypeScript types
- Strict Zod validation
- Small focused modules

- Server-only AI logic
- Descriptive naming
- Reusable UI components only where beneficial
- Pure scoring and validation utilities
- Abort or timeout handling for external requests
- Helpful user-facing errors
- Accessible controls
- Stable React keys
- No TypeScript `any` unless unavoidable and justified

After implementation, always run:

```
npm run lint
npm run typecheck
npm test
npm run build
```

If the generated project does not initially include all these scripts, propose the smallest appropriate scripts in the plan.

Do not suppress legitimate errors merely to make the build pass.

14. SECURITY AND DATA BOUNDARIES

- Keep OpenAI and sponsor API calls server-side.
 - Do not log secrets.
 - Do not commit `.env`.
 - Treat imported comments as untrusted text.
 - Prevent prompt injection contained inside comments from changing system behaviour.
 - Clearly delimit comments as data, not instructions.
 - Limit input size.
 - Validate all request bodies.
 - Avoid rendering unsanitized HTML.
 - Add basic request timeout handling.
 - Do not introduce unnecessary persistent storage.
-

15. WHAT THE FINAL PLAN MUST CONTAIN

Before writing any code, return one implementation plan containing:

A. Repository assessment

- Existing files and structure
- Whether the repo is empty or partially implemented
- Problems or conflicts found

B. Final architecture

- Browser-to-server data flow
- Route handlers
- LangGraph state
- Graph nodes and edges
- Which nodes use LLM calls
- Which nodes are deterministic
- Retry and fallback flow

C. Exact file tree

Show every file you expect to create or modify.

D. Data contracts

Define the planned Zod schemas and major TypeScript types.

E. Scoring formulas

Provide the planned deterministic formulas and inputs.

F. UI plan

- Page structure
- Components
- Interaction flow
- Loading/error/fallback states
- Before/after comparison

G. API plan

- Request shape
- Response shape
- Server boundaries
- Environment variables
- Timeout and error strategy

H. Model plan

- Exact currently available model choices you recommend
- Why
- Which call uses which model
- Estimated number of calls per analysis
- How structured output will be enforced

I. Dependency list

For every dependency:

- Name
- Purpose
- Whether it is essential
- Whether a simpler native alternative exists

J. Test plan

- Unit tests
- Schema tests
- Evidence-integrity tests
- Scoring tests
- API fallback tests
- Browser smoke-test sequence

K. Build order

A numbered sequence designed to reach a stable vertical slice quickly.

For each phase include:

- Goal
- Files affected
- Verification command
- Runnable checkpoint

L. Scope control

Clearly label:

- Must have
- Should have
- Stretch
- Explicitly excluded

M. Risk register

List the largest implementation and demo risks, including:

- Model latency
- Invalid structured output
- Fabricated evidence IDs
- Overly agreeable audience segments
- UI incompleteness
- Deployment failure
- External service failure

For each, give a concrete mitigation.

N. Time-saving decisions

Identify anything from this request that you recommend simplifying to protect the final demonstration. Do not expand the scope merely because the specification is detailed.

O. Approval checkpoint

End with:

1. A compact summary of the proposed implementation.
2. The first implementation phase you will execute after approval.
3. Any truly blocking decisions requiring my confirmation.
4. A clear statement that no implementation has yet been performed.

16. FINAL PRIORITY ORDER

When trade-offs arise, optimise in this exact order:

1. Complete working vertical slice
2. Evidence credibility
3. Before-versus-after demo
4. Reliable agentic scenario workflow
5. Deterministic diagnostics
6. Strong UI
7. Deployment
8. Sponsor integration
9. Additional features

A polished implementation of one complete workflow is preferable to a larger half-working application.

Do not begin coding. Produce the final reviewable plan now.